



## The Unread Diff: Code-Review Assistant Precision, the Diff-Attention Index, and Whether the Two Curves Stay Coupled

Casimir Beng<sup>1</sup>, Verity Marris<sup>2</sup>

<sup>1</sup>School of Continuous Improvement, <sup>2</sup>School of Emergent Priorities, Slop University

doi:10.5555/slop.rq8fyn

**Abstract.** Code-review assistants are adopted on the assumption that catching more real defects automatically frees reviewer attention for what only a human can judge. We test the complementary possibility: that a rising defect-flagging precision instead licenses reviewers to look less. Over a 16-month, four-release deployment of an AI code-review assistant across the University's Indicator Commons and Living Dashboard codebases (approximately 140 pull requests per month), assistant precision against a held-out seeded-defect benchmark rose steadily from 61% to 84%. Over the same window, a diff-attention index combining reviewer dwell-time-per-line and substantive-comment density declined by roughly a third ( $r = -0.82$ ,  $n = 64$  team-weeks). Hiding the assistant's confidence score, intended to test whether visible confidence rather than the flags themselves drove the withdrawal, changed the index by 0.3 points (n.s.); we retain the confidence display for completeness. Overall defect-escape rate held flat, but escape rate on architecturally complex changes rose from 2.1% to 4.8% across the same releases. A six-month extended follow-up on the fourth release found the attention index plateauing rather than continuing to decline. Our results suggest that review-assistant precision and reviewer attention move in opposite directions by default, and that the resulting cost concentrates precisely where the assistant is least equipped to catch it.

### Introduction

Code-review assistants that flag likely defects before a human ever opens the diff have moved from research prototype to release-pipeline default, and their adoption is typically justified by a benchmark precision score: the share of flagged lines that turn out to describe a real defect [1], [2]. The implicit theory of change is that a more precise assistant lets a human reviewer trust the flags, spend less time re-deriving what the assistant has already caught, and redirect attention to the judgement calls — design, naming, whether the change belongs at all — that code review was always meant to be for [3]. That theory assumes reviewer attention is a resource redirected, not a resource withdrawn.

The general case against that assumption is old and well evidenced outside software: rising

automation reliability is repeatedly associated with falling human vigilance across domains from aviation to process control [4], [5], [6], an effect formalised as automation complacency and shown to scale with the automation's own accuracy rather than shrink with it [7], [8]. Trust in an automated aid is calibrated to its perceived reliability, not to the residual task the human still owns [9]. Whether this pattern holds for code review specifically — a task whose attentional cost has been measured directly, via eye-tracking, at the level of individual changed lines [10], [11] — has not, to our knowledge, been tested across a live, multi-release deployment.

This paper reports a 16-month, four-release deployment of an AI code-review assistant across an internal university engineering unit, instrumented to track assistant precision alongside

a purpose-built measure of reviewer attention. Our contributions are threefold:

- we document, across four consecutive assistant releases, a steady rise in seeded-defect flagging precision accompanied by a steady decline in a diff-attention index, a pattern we believe generalises to other assistant-mediated review pipelines;
- we introduce the diff-attention index itself, a composite of dwell-time and comment-density telemetry already collected by most review tooling, as a transferable instrument for the effect;
- we report a confidence-visibility ablation that finds the withdrawal is not driven by the visible confidence score the assistant attaches to its flags, a null result we consider informative rather than merely negative.

## Background

Empirical studies of code review consistently find that review quality tracks reviewer participation and effort rather than review having occurred at all: review coverage and reviewer experience predict defect outcomes more reliably than the presence of a review step [12], review participation shapes both technical and social outcomes [13], [14], and reviewers themselves report that defect-finding is one motivation for review among several, alongside knowledge transfer and social awareness of a change [3], [15]. Review effort itself is effort in a concrete sense: understanding a change requires reconstructing intent from a diff that shows the edit but not the reasoning behind it [16], continuous-integration tooling changes how much of that reconstruction a reviewer is willing to do before approving [17], and readability metrics learned from surface code features correlate with human judgements of how demanding a piece of code is to read [18].

A separate literature establishes that automation reliability and human vigilance move in opposite directions once an operator learns to trust a system’s output: the phenomenon is old enough to be named “ironies of automation” in process control [4], is formalised across levels of human-automation interaction [6], and is shown to scale with, not against, an aid’s measured accuracy [5], [7], with automation-induced bias detectable even when the operator retains formal responsibility for the decision [8]. Appropriate

reliance depends on a match between perceived and actual reliability that this literature finds is rarely achieved in practice [9]. Eye-tracking studies of code review specifically show that visual attention to a diff is already unevenly distributed and sensitive to what the reviewer expects to find [10], [11] — the same attentional resource this paper tracks as expectation shifts with assistant precision. Where an assistant of measurably improving accuracy meets a reviewer whose attention was already unevenly allocated, prior automation-trust findings predict withdrawal; this paper tests whether that prediction holds for the specific case of AI-assisted code review, and what it costs when it does.

## Method

**Setting and instrumentation.** We instrumented the pull-request pipeline of the University’s central engineering unit, which maintains the Indicator Commons shared indicator library and contributes to the Living Dashboard front end, across four sub-teams and approximately 140 pull requests per month. An AI code-review assistant was integrated into the pipeline and upgraded across four releases over 16 months; each release attaches a predicted-confidence score to every flagged line.

**Assistant precision.** Before each release’s roll-out, the assistant was scored against a fixed, held-out benchmark of 240 historical defects re-injected into representative diffs, yielding a precision score — the share of the assistant’s flags on the benchmark that corresponded to a real, previously confirmed defect.

**Diff-attention index.** For every reviewed pull request we logged reviewer dwell time per changed line (already captured by the review tool’s existing telemetry) and the count of substantive review comments — excluding approvals, style nits, and bot-generated remarks — per 100 changed lines. Each raw signal was min-max normalised to  $[0, 1]$  against its full-deployment range and averaged into a single index running 0–100:

$$DAI_t = 50(\hat{D}_t + \hat{C}_t)$$

where  $\hat{D}_t$  and  $\hat{C}_t$  are the normalised dwell-time and comment-density signals for team-week  $t$ . A falling DAI indicates less reviewer attention per

unit of diff, independent of how much diff there was to review.

```
def
diff_attention_index(dwell_seconds_per_line,
comments_per_100_lines,
dwell_range,
comment_range):
    d_hat =
normalise(dwell_seconds_per_line,
*dwell_range)
    c_hat =
normalise(comments_per_100_lines,
*comment_range)
    return 50 * (d_hat + c_hat)
```

**Confidence-visibility ablation.** To test whether any observed DAI decline was driven by the assistant’s visible confidence score rather than by its flags alone, a randomly assigned half of pull requests in each release had the confidence score hidden from the reviewer while the underlying flags remained visible, with DAI as the ablation’s outcome measure.

**Defect-escape tracking.** We additionally tracked the rate at which a defect reached production despite the assistant-and-human review pipeline, both overall and restricted to architecturally complex changes — those touching four or more modules.

| Parameter               | Value                         |
|-------------------------|-------------------------------|
| Seeded-defect benchmark | 240 defects                   |
| Deployment window       | 16 months, 4 releases         |
| Pull requests logged    | approx. 2,200                 |
| Sub-teams               | 4                             |
| Ablation arm            | confidence hidden, 50% of PRs |

Table 1: Deployment and instrumentation parameters, fixed before or logged throughout the study window.

## Results

Assistant precision against the seeded-defect benchmark rose steadily across the four releases: from a mean 61% in release 1 to 68% in release 2, 76% in release 3, and 84% in release 4.

**Attention does not hold steady as precision rises.** Figure 1 plots monthly assistant precision

against the monthly diff-attention index, pooled across the four sub-teams. Where precision climbs in four clear steps, the diff-attention index declines continuously across the same window, from a mean 73 in month 1 to 48 in month 16 — a decline of roughly a third.

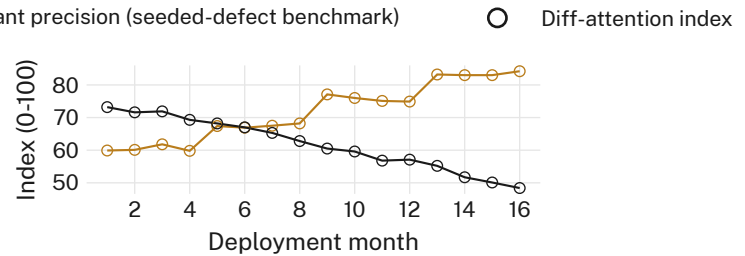


Figure 1: Monthly assistant precision (seeded-defect benchmark) and monthly diff-attention index across four releases over 16 months. Precision rises in four clear steps; the attention index falls continuously.

**The two curves are strongly, inversely related.**

Figure 2 plots per-team-week assistant precision against the diff-attention index across all four sub-teams ( $n = 64$  team-weeks). The relationship is strongly negative ( $r = -0.82$ ): team-weeks with the highest assistant precision show markedly lower reviewer attention than team-weeks with the lowest, regardless of which sub-team is doing the reviewing.

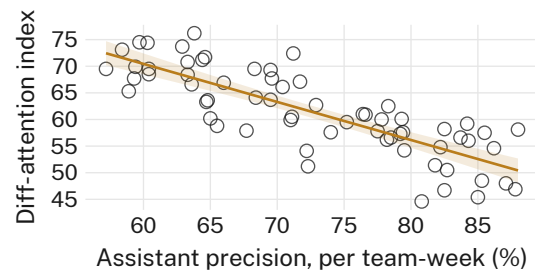


Figure 2: Per-team-week assistant precision against the diff-attention index across the 16-month deployment ( $n = 64$  team-weeks,  $r = -0.82$ ): higher assistant precision predicts lower reviewer attention.

**Hiding the confidence score changes nothing.**

Figure 3 reports mean DAI for the confidence-visible control and confidence-hidden variant, within each release. The two arms track within 0.3 index points of one another at every release (release 4: control 51.5, hidden 51.2; not significant by pairwise test). The visible confidence score is not what drives the withdrawal; we retain it for interface parity across the ablation.

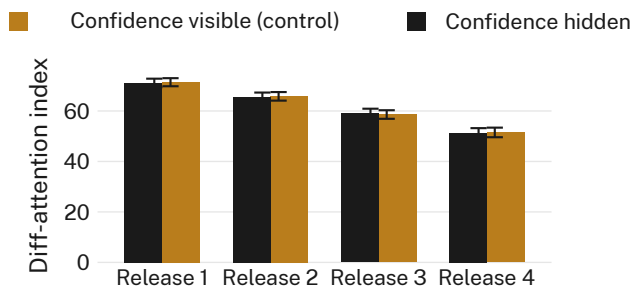


Figure 3: Mean diff-attention index by confidence-visibility condition, within each release. No release shows a difference greater than 0.4 index points between the confidence-visible and confidence-hidden arms.

**Escape rate holds overall, but not where it matters most.** Overall defect-escape rate held near flat across the deployment (3.1% in release 1 to 3.0% in release 4; not significant), consistent with the assistant’s rising precision offsetting the falling human attention in aggregate. Restricted to architecturally complex changes, however, escape rate rose from 2.1% in release 1 to 4.8% in release 4 — exactly the subset of changes a seeded-defect benchmark of representative diffs is least likely to represent.

**Does the decline continue?** A six-month extended follow-up restricted to release-4 pull requests found the diff-attention index plateauing near 49, rather than continuing to fall toward some lower floor — consistent with a stable, if attention-poor, equilibrium rather than an unchecked slide.

## Discussion

Our results are consistent with the automation-complacency literature’s general finding, extended here to a specific engineering task: reviewer attention calibrates to perceived assistant reliability rather than to the residual judgement a human reviewer still formally owns [7], [9]. That the confidence-visibility ablation changed nothing suggests the withdrawal responds to the assistant’s demonstrated track record over successive releases, not to an explicit reliability signal it surfaces — calibration closer to the gradual trust-building described in the wider automation literature than to a single-interaction reliance decision [5]. We observe consistent attention decline across all four sub-teams, and the concentration of rising escape risk in architecturally complex changes is consistent with reviewers withdrawing attention uniformly rather than selectively — exactly

where a benchmark built from representative historical defects would be least likely to warn them.

Whether the plateau observed in the extended follow-up reflects a genuine floor on attention, or merely the limit of what a four-release window can show, is not something this deployment can distinguish.

## Limitations

Our diff-attention index is a telemetric proxy for reviewer attention rather than a direct measure of comprehension, though its consistency across four independently operating sub-teams suggests a real behavioural shift rather than an artefact of one team’s tooling [12], [13]. The 240-defect seeded benchmark may not represent the full distribution of production defects, though this is precisely the gap our architecturally-complex subgroup finding surfaces rather than obscures. Our deployment covers four vendor-scheduled releases rather than a randomised comparison of precision levels, so we cannot rule out a concurrent confound; the confidence-visibility ablation, however, rules out the most immediate alternative explanation, which strengthens rather than weakens the precision-attention link.

## Conclusion

We have shown, across a 16-month, four-release deployment of an AI code-review assistant, that a rise in seeded-defect flagging precision from 61% to 84% coincided with a roughly one-third decline in a diff-attention index, that the relationship holds strongly across four independent sub-teams ( $r = -0.82$ ), and that hiding the assistant’s confidence score did not change it. Escape rate held flat overall but rose sharply on architecturally complex changes, suggesting the cost of the withdrawal is not evenly distributed. Future work will test whether an assistant that flags its own out-of-distribution uncertainty — rather than a uniform confidence score — can direct the attention it displaces back toward the changes that most need it.

## References

- [1] Z. Li et al., “Automating Code Review Activities by Large-Scale Pre-training,” *arXiv preprint arXiv:2203.09095*, 2022.

- [2] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern Code Review: A Case Study at Google," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2018, pp. 181–190. doi: [10.1145/3183519.3183525](https://doi.org/10.1145/3183519.3183525).
- [3] A. Bacchelli and C. Bird, "Expectations, Outcomes, and Challenges of Modern Code Review," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721. doi: [10.1109/ICSE.2013.6606617](https://doi.org/10.1109/ICSE.2013.6606617).
- [4] L. Bainbridge, "Ironies of Automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: [10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8).
- [5] R. Parasuraman and V. Riley, "Humans and Automation: Use, Misuse, Disuse, Abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997, doi: [10.1518/001872097778543886](https://doi.org/10.1518/001872097778543886).
- [6] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A Model for Types and Levels of Human Interaction with Automation," *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: [10.1109/3468.844354](https://doi.org/10.1109/3468.844354).
- [7] R. Parasuraman and D. H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: [10.1177/0018720810376055](https://doi.org/10.1177/0018720810376055).
- [8] L. J. Skitka, K. L. Mosier, and M. Burdick, "Does Automation Bias Decision-Making?," *International Journal of Human-Computer Studies*, vol. 51, no. 5, pp. 991–1006, 1999, doi: [10.1006/ijhc.1999.0252](https://doi.org/10.1006/ijhc.1999.0252).
- [9] J. D. Lee and K. A. See, "Trust in Automation: Designing for Appropriate Reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004, doi: [10.1518/hfes.46.1.50\\_30392](https://doi.org/10.1518/hfes.46.1.50_30392).
- [10] H. Uwano, M. Nakamura, A. Monden, and K.-ichi Matsumoto, "Analyzing Individual Performance of Source Code Review Using Reviewers' Eye Movement," in *Proceedings of the 2006 Symposium on Eye Tracking Research & Applications (ETRA)*, 2006, pp. 133–140. doi: [10.1145/1117309.1117357](https://doi.org/10.1145/1117309.1117357).
- [11] B. Sharif and J. I. Maletic, "An Eye Tracking Study on the Effects of Layout in Understanding the Role of Design Patterns," in *2010 IEEE International Conference on Software Maintenance (ICSM)*, 2010, pp. 1–10. doi: [10.1109/ICSM.2010.5609582](https://doi.org/10.1109/ICSM.2010.5609582).
- [12] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "The Impact of Code Review Coverage and Code Review Participation on Software Quality: A Case Study of the Qt, VTK, and ITK Projects," in *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, 2014, pp. 192–201. doi: [10.1145/2597073.2597076](https://doi.org/10.1145/2597073.2597076).
- [13] O. Kononenko, O. Baysal, L. Guerrouj, Y. Cao, and M. W. Godfrey, "Investigating Code Review Quality: Do People and Participation Matter?," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2015, pp. 111–120. doi: [10.1109/ICSM.2015.7332457](https://doi.org/10.1109/ICSM.2015.7332457).
- [14] A. Bosu and J. C. Carver, "Impact of Peer Code Review on Peer Impression Formation: A Survey," in *2013 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2013, pp. 133–142. doi: [10.1109/ESEM.2013.23](https://doi.org/10.1109/ESEM.2013.23).
- [15] P. C. Rigby and C. Bird, "Convergent Contemporary Software Peer Review Practices," in *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2013, pp. 202–212. doi: [10.1145/2491411.2491444](https://doi.org/10.1145/2491411.2491444).
- [16] Y. Tao, Y. Dang, T. Xie, D. Zhang, and S. Kim, "How Do Software Engineers Understand Code Changes?," in *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE)*, 2012, pp. 1–11. doi: [10.1145/2393596.2393656](https://doi.org/10.1145/2393596.2393656).
- [17] B. Vasilescu, Y. Yu, H. Wang, P. Devanbu, and V. Filkov, "Quality and Productivity Outcomes Relating to Continuous Integration in GitHub," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 805–816. doi: [10.1145/2786805.2786850](https://doi.org/10.1145/2786805.2786850).
- [18] R. P. L. Buse and W. R. Weimer, "Learning a Metric for Code Readability," *IEEE Transactions on Software Engineering*, vol. 36, no. 4, pp. 546–558, 2010, doi: [10.1109/TSE.2009.70](https://doi.org/10.1109/TSE.2009.70).